

Série TP 5 : Fonctions et Procédures

Corrigé

Exercice 1:

Soit une base de données dont le schéma est le suivant :

Table	Colonne	Type	Description
EMPLOYE	N_EMP	VARCHAR2(4)	Numéro employé
	NOM_EMP	VARCHAR2(20)	Nom employé
	QUALIF_EMP	VARCHAR2(12)	Qualification employé
TRANSPORTER	N_TRANPORTE	VARCHAR2(5)	Numéro transport
	N_VISITE	VARCHAR2(5)	Numéro visite
	N_EMPLOYE	VARCHAR2(4)	Numéro employé
CHANTIER	N_CHANTIER	VARCHAR2(10)	Numéro chantier
	NOM_CH	VARCHAR2(10)	Nom chantier
	ADRESSE_CH	VARCHAR2(15)	Adresse chantier
VEHICULE	N_VÉHICULE	VARCHAR2(10)	Numéro véhicule
	TYPE_VÉHICULE	VARCHAR2(11)	Type véhicule
	KILOMETRAGE	NUMBER	Kilométrage véhicule
VISITE	N_VISITE	VARCHAR2(5)	Numéro visite
	N_CHANTIER	VARCHAR2(10)	Numéro chantier
	N_VÉHICULE	VARCHAR2(10)	Numéro véhicule
	KILOMETRES	NUMBER	Kilomètres parcourus
	DATE_JOUR	DATE	Date de la visite
	N_CONDUCTEUR	VARCHAR2(4)	Numéro conducteur

Travail à faire :

1. Ecrire en PL/SQL la procédure **VisiteChantier** permettant pour un chantier, d'afficher le nombre de visites et les détails de toutes les visites sous la forme suivante :

- Nombre de visites : nbrVisites
- Numéro de visite : N_VISITE, Véhicule : N_VEHICULE, Conducteur : N_CONDUCTEUR, Date : DATE_JOUR

```
CREATE OR REPLACE PROCEDURE VisiteChantier(p_n_chantier IN
CHANTIER.N_CHANTIER%TYPE)
IS
    v_nombre_visites NUMBER;
    CURSOR c_visites IS
        SELECT v.N_VISITE, v.N_VEHICULE, v.N_CONDUCTEUR, v.DATE_JOUR
        FROM VISITE v
        WHERE v.N_CHANTIER = p_n_chantier;
BEGIN
    SELECT COUNT(*)
    INTO v_nombre_visites
    FROM VISITE
    WHERE N_CHANTIER = p_n_chantier;
    DBMS_OUTPUT.PUT_LINE('Nombre de visites : ' || v_nombre_visites);
    FOR visite_rec IN c_visites LOOP
        DBMS_OUTPUT.PUT_LINE(
            'Numéro de visite : ' || visite_rec.N_VISITE ||
            ', Véhicule : ' || visite_rec.N_VEHICULE ||
            ', Conducteur : ' || visite_rec.N_CONDUCTEUR ||
            ', Date : ' || TO_CHAR(visite_rec.DATE_JOUR, 'DD/MM/YYYY')
        );
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Aucune visite trouvée pour le chantier ' || p_n_chantier);
```

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Erreur : ' || SQLERRM);

END VisiteChantier;

/

2. Ecrire la procédure **finAnnee** permettant de rajouter à chaque véhicule les kilométrages faits lors des visites de l'année.

Pour ceci, on utilisera un seul curseur pour parcourir tous les véhicules. Il faudra ensuite supprimer toutes les missions de l'année (visites et détails des trajets des employés transportés)

Penser à utiliser la fonction Oracle SQL NVL(expression, remplacement) qui permet de gérer les valeurs NULL.

Exemple : SELECT NVL(salaire, 0) FROM employees;

- Si salaire est NULL → retourne 0
- Si salaire vaut 5000 → retourne 5000

CREATE OR REPLACE PROCEDURE finAnnee(p_annee IN NUMBER)

IS

CURSOR c_vehicules_visites IS

SELECT v.N_VEHICULE, SUM(v.KILOMETRES) as total_km

FROM VISITE v

WHERE EXTRACT(YEAR FROM v.DATE_JOUR) = p_annee

GROUP BY v.N_VEHICULE;

v_count_visites NUMBER;

v_count_transport NUMBER;

BEGIN

SELECT COUNT(*)

INTO v_count_visites

FROM VISITE

WHERE EXTRACT(YEAR FROM DATE_JOUR) = p_annee;

IF v_count_visites = 0 THEN

DBMS_OUTPUT.PUT_LINE('Aucune visite trouvée pour l'année ' || p_annee);

```
RETURN;  
END IF;  
FOR vehicule_rec IN c_vehicules_visites LOOP  
    UPDATE VEHICULE  
    SET KILOMETRAGE = KILOMETRAGE + vehicule_rec.total_km  
    WHERE N_VEHICULE = vehicule_rec.N_VEHICULE;  
  
    DBMS_OUTPUT.PUT_LINE('Véhicule ' || vehicule_rec.N_VEHICULE ||  
        ' : ' || vehicule_rec.total_km || ' km ajoutés');  
END LOOP;  
  
SELECT COUNT(*)  
INTO v_count_transport  
FROM TRANSPORTER t  
JOIN VISITE v ON t.N_VISITE = v.N_VISITE  
WHERE EXTRACT(YEAR FROM v.DATE_JOUR) = p_annee;  
  
IF v_count_transport > 0 THEN  
    DELETE FROM TRANSPORTER t  
    WHERE EXISTS (  
        SELECT 1  
        FROM VISITE v  
        WHERE v.N_VISITE = t.N_VISITE  
        AND EXTRACT(YEAR FROM v.DATE_JOUR) = p_annee  
    );  
    DBMS_OUTPUT.PUT_LINE(v_count_transport || ' transports supprimés');  
END IF;  
  
DELETE FROM VISITE  
WHERE EXTRACT(YEAR FROM DATE_JOUR) = p_annee;
```

```
DBMS_OUTPUT.PUT_LINE(v_count_visites || ' visites supprimées');
```

```
COMMIT;
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
```

```
DBMS_OUTPUT.PUT_LINE('ERREUR : ' || SQLERRM);
```

```
END finAnnee;
```

```
/
```

Exercice 2 :

Soit une base de données dont le schéma est le suivant (les clés primaires des relations sont soulignées).

- Employé (NumEmp NUMBER, NomEmp VARCHAR(20), PrenomEmp VARCHAR(20), FonctionEmp VARCHAR(20), SalaireEmp NUMBER, VilleEmp VARCHAR(20))
- GradeSalaire(Grade VARCHAR(20), SalaireMin NUMBER, SalaireMax NUMBER)
- Département (NumDep NUMBER, NomDep VARCHAR(20), BudgetDep NUMBER, VilleDep VARCHAR(20))
- DirigerDepartement (#NumDep NUMBER, #NumEmp NUMBER, DateDebut DATE, DateFin DATE)
- Projet (NumProjet NUMBER, TitreProjet VARCHAR(100), Montant NUMBER, DateDebut DATE, DateFin DATE)
- Departement_Projet (#NumProjet NUMBER, #NumDep NUMBER, MontantParticipation NUMBER)

Travail à faire :

Ecrire la procédure Participer_Projet permettant la participation de tous les départements à la réalisation d'un nouveau projet (Montant_Réalisation/Nombre_de_départements). La transaction doit enregistrer dans un premier temps le nouveau projet en prenant en considération les exceptions suivantes :

- NumProjet existe déjà
- Montant_Participation dépasse le montant restant du budget de département

```
CREATE OR REPLACE PROCEDURE Participer_Projet(
```

```
p_NumProjet IN Projet.NumProjet%TYPE,  
p_TitreProjet IN Projet.TitreProjet%TYPE,  
p_Montant IN Projet.Montant%TYPE,  
p_DateDebut IN Projet.DateDebut%TYPE,  
p_DateFin IN Projet.DateFin%TYPE  
)  
IS  
  
v_nombre_departements NUMBER;  
v_montant_participation NUMBER;  
v_budget_restant NUMBER;  
v_projet_existe NUMBER;  
  
CURSOR c_departements IS  
    SELECT NumDep, BudgetDep  
    FROM Departement;  
  
projet_deja_existe EXCEPTION;  
budget_insuffisant EXCEPTION;  
PRAGMA EXCEPTION_INIT(projet_deja_existe, -20001);  
PRAGMA EXCEPTION_INIT(budget_insuffisant, -20002);  
  
BEGIN  
    SELECT COUNT(*)  
    INTO v_projet_existe  
    FROM Projet  
    WHERE NumProjet = p_NumProjet;  
  
    IF v_projet_existe > 0 THEN  
        RAISE projet_deja_existe;  
    END IF;  
    SELECT COUNT(*)
```

```
INTO v_nombre_departements
FROM Departement;

IF v_nombre_departements = 0 THEN
    RAISE_APPLICATION_ERROR(-20003, 'Aucun département trouvé dans la base de données');
END IF;

v_montant_participation := p_Montant / v_nombre_departements;
INSERT INTO Projet (NumProjet, TitreProjet, Montant, DateDebut, DateFin)
VALUES (p_NumProjet, p_TitreProjet, p_Montant, p_DateDebut, p_DateFin);

DBMS_OUTPUT.PUT_LINE('Projet ' || p_NumProjet || ' créé avec succès');
DBMS_OUTPUT.PUT_LINE('Montant total : ' || p_Montant);
DBMS_OUTPUT.PUT_LINE('Nombre de départements : ' || v_nombre_departements);
DBMS_OUTPUT.PUT_LINE('Montant par département : ' || v_montant_participation);

FOR dep_rec IN c_departements LOOP
    -- Vérifier le budget restant du département
    SELECT BudgetDep
    INTO v_budget_restant
    FROM Departement
    WHERE NumDep = dep_rec.NumDep;

    IF v_budget_restant < v_montant_participation THEN
        RAISE_APPLICATION_ERROR(-20002,
            'Budget insuffisant pour le département ' || dep_rec.NumDep);
    END IF;

    INSERT INTO Departement_Projet (NumProjet, NumDep, MontantParticipation)
    VALUES (p_NumProjet, dep_rec.NumDep, v_montant_participation);
```

```
-- Mettre à jour le budget du département
UPDATE Departement
SET BudgetDep = BudgetDep - v_montant_participation
WHERE NumDep = dep_rec.NumDep;

DBMS_OUTPUT.PUT_LINE('Département ' || dep_rec.NumDep ||
    ' associé au projet avec une participation de ' ||
    v_montant_participation);
END LOOP;

COMMIT;

EXCEPTION
    WHEN projet_deja_existe THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERREUR : Le projet ' || p_NumProjet || ' existe déjà');
    WHEN budget_insuffisant THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERREUR : Budget insuffisant pour un département');
    WHEN ZERO_DIVIDE THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERREUR : Division par zéro - aucun département trouvé');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('ERREUR : ' || SQLERRM);
END Participer_Projet;
/
```